EDUFRAMEWORK DOCUMENTATION

DEVELOPMENT GUIDE

Setting Up the Development Environment and Creating an EduFramework Project

PlatformIO · S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1 Introduction

EduFramework is a software development ecosystem for the NXP S32K144 microcontroller, designed to simplify application development through higher-level APIs and PlatformIO integration. The ecosystem provides a unified workflow from project organization and source-code development to building, uploading, and hardware debugging.
This document provides step-by-step instructions for setting up the development environment and creating an EduFramework project for the MaaZEDU board. After completing the setup, the environment can be used to continue with the EduFramework Laboratory Series or to develop custom applications for the S32K144.
The EduFramework ecosystem is organized into three main components: **EduFramework**, **Platform-NXPS32K**, and **EduFramework Laboratory Series**. Each component has a distinct role in the development workflow.

Component	Role
EduFramework	Application-development software layer that includes Arduino-style APIs, logical pin definitions, and device libraries for the S32K144.
Platform-NXPS32K	Integrates the S32K144 and EduFramework with PlatformIO, supporting project configuration, build, upload, and debugging.
EduFramework Laboratory Series	A collection of laboratory exercises, example projects, and documentation for using EduFramework by topic.

Table 1.1. Main components of the EduFramework ecosystem

EduFramework is the software component used directly by applications. Its Arduino-style APIs simplify common microcontroller operations, while device libraries support external modules and sensors. The framework repository also contains low-level driver source code and other components required for studying or further developing the framework.

GitHub Repository `EduFramework Project`

Platform-NXPS32K integrates the S32K144 with the PlatformIO ecosystem. The platform defines the board, framework, toolchain, and related tools so that PlatformIO can build, upload, and debug EduFramework projects. Platform definitions and configuration can be referenced directly in the project repository.

GitHub Repository `Platform-NXPS32K`

The EduFramework Laboratory Series provides topic-based laboratory exercises built on EduFramework and Platform-NXPS32K. The laboratories are designed to introduce the framework APIs progressively and apply them to hardware, providing a foundation for developing custom S32K144 applications. The Laboratory Series repository contains laboratory projects, completed documents, and the source files used to develop the documentation.

GitHub Repository `EduFramework Labs and Documentation`

2 Setting Up the Development Environment

Before creating an EduFramework project, the required development tools must be prepared on the computer. The environment used in this guide consists of Visual Studio Code, Git, and PlatformIO IDE. This section explains how to install and verify each component before creating a project.

2.1 Installing Visual Studio Code

Visual Studio Code (VS Code) is used as the source-code editor and as the environment that hosts PlatformIO for EduFramework development.

Open the official Visual Studio Code website:

Official Website	`Visual Studio Code`

Download the Visual Studio Code version appropriate for the operating system and install it according to the instructions on the official website. After installation, launch Visual Studio Code to verify that the application starts correctly.

Note

The Visual Studio Code interface and installation process may vary depending on the operating system and software version. The Development Guide does not require any special Visual Studio Code installation options.

2.2 Installing and Configuring Git

Git is used for source-code management and allows PlatformIO to access EduFramework components hosted on GitHub. In addition to installing Git, user information must be configured so that Git can record the author of commits created on the computer.

2.2.1 Downloading and Installing Git

Open the official Git installation page:

Installation Page	`Git - Install`

On the installation page, select the Git version appropriate for the operating system.

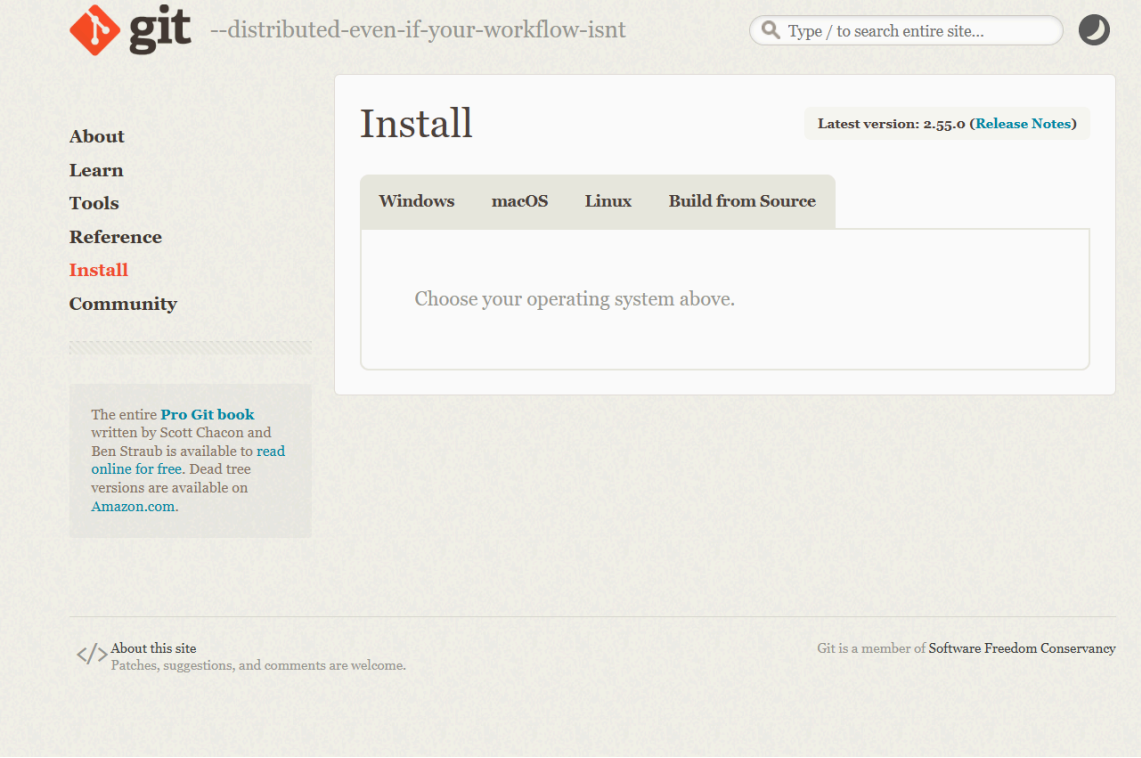

Figure 2.1. Official Git installation page

After selecting the operating system, download the installer appropriate for the system. The following figure illustrates selecting a Git installer for Windows x64.

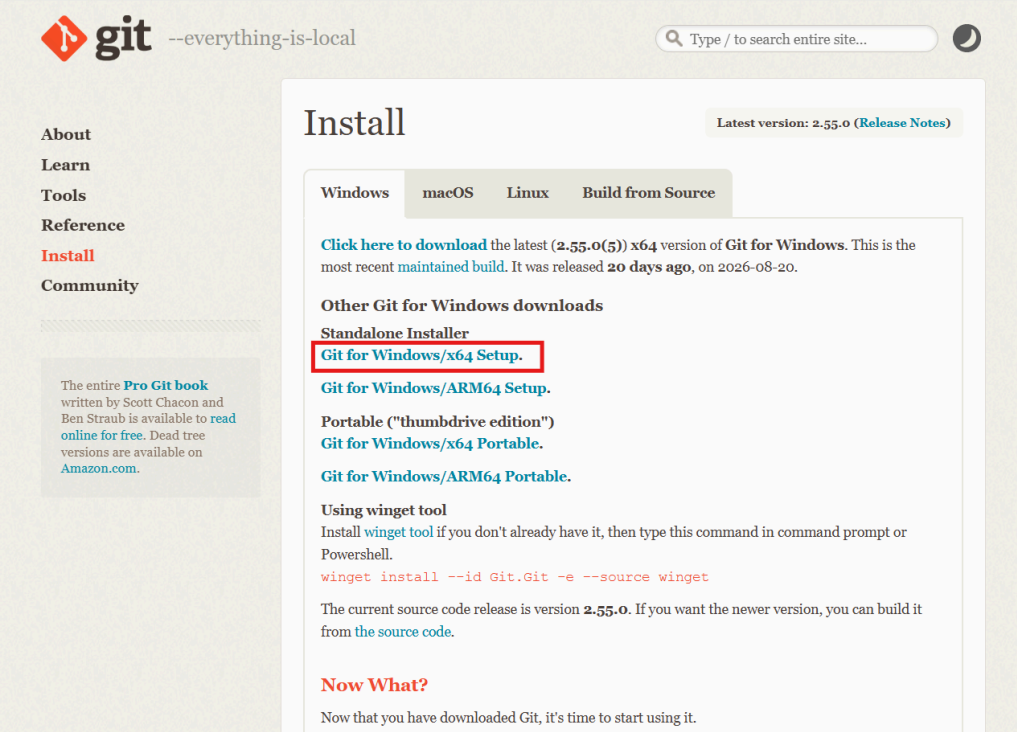

Figure 2.2. Example of selecting a Git installer on Windows

Install Git using the downloaded installer. The installer default options can be retained unless a specific configuration is required.

For Git for Windows, when the installer asks for the default Git editor, select **Visual Studio Code** so that VS Code is used for Git operations that require an editor.

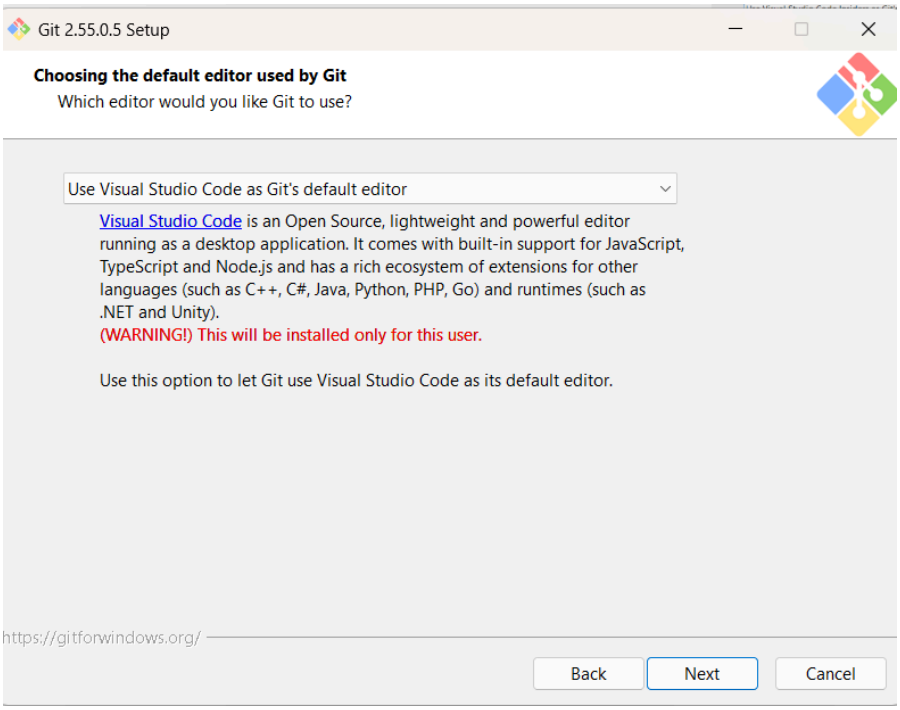

Figure 2.3. Selecting Visual Studio Code as the default Git editor

Continue the installation using the default options until it is complete.

2.2.2 Verifying the Git Installation

After installation, open a terminal and run:

Terminal

```
git --version
```

If Git is installed and accessible from the terminal, the output will display the Git version available on the system.

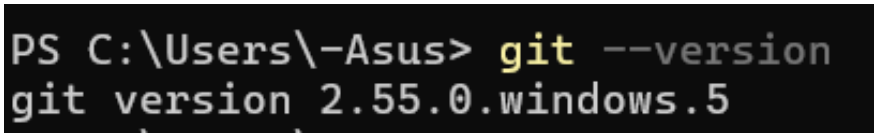

Figure 2.4. Verifying the Git version after installation

Note

The displayed version number may differ from the figure. The Git installation is successful as long as the `git --version` command returns valid version information.

2.2.3 Configuring Git User Information

Git uses `user.name` and `user.email` to identify the author recorded in each commit. When using GitHub to host repositories, it is recommended to use an email address associated with the GitHub account so that commits can be attributed to the corresponding account.
If a GitHub account is not available, one can be created on the official website:

Official Website GitHub

Account information and email addresses can be checked on GitHub before configuring Git. The following figure illustrates where the account information can be found on GitHub.

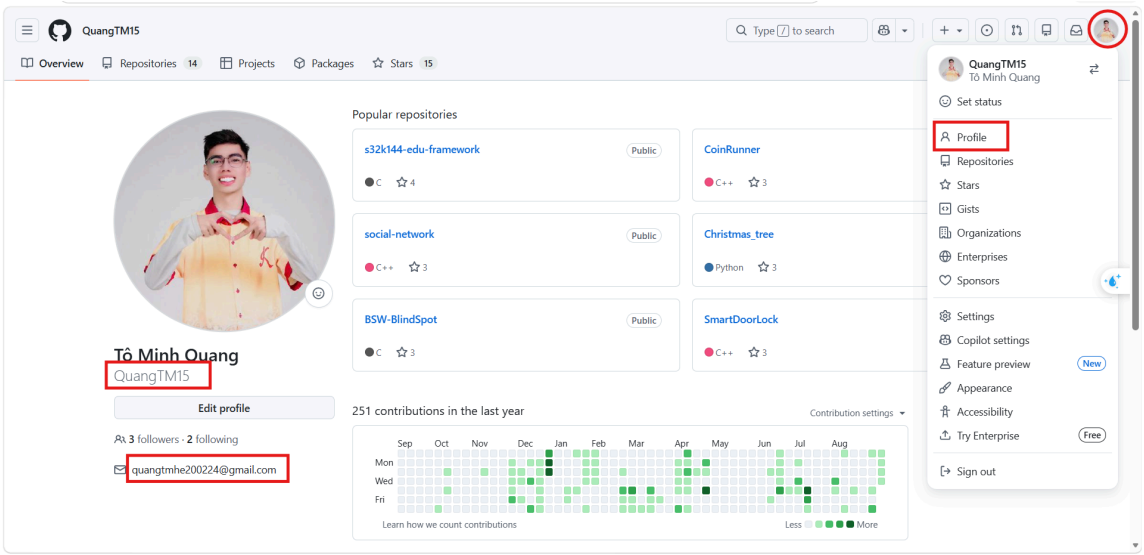

Figure 2.5. Example of GitHub account and email information

Open a terminal and configure the name used for commits with:

Terminal

```
git config --global user.name "Your Name"
```

Next, configure the email address:

Terminal

```
git config --global user.email "your-email@example.com"
```

Here, `Your Name` is the name recorded in commits and `your-email\@example.com` is the user email address.
The `user.name` value does not have to match the GitHub username.
The `--global` option applies these values to Git repositories for the current user account on the computer.
After configuration, verify the settings with:

Terminal

```
git config --global --list
```

The output should contain the configured `user.name` and `user.email` values.

Git global configuration

```
user.name=Your Name user.email=your-email@example.com
```

The following figure illustrates the result after Git has been configured successfully.

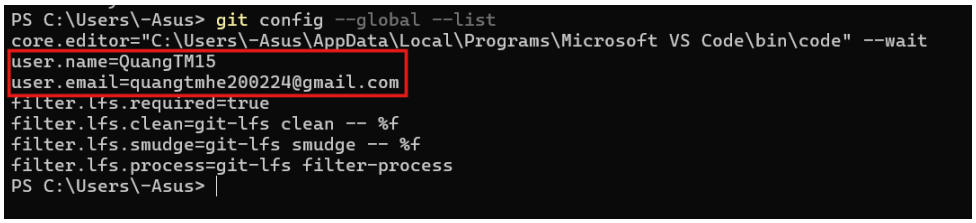

Figure 2.6. Verifying the Git user configuration

EXPECTED RESULT

Git has been installed and configured successfully. The `git --version` command can be executed from the terminal, and the output of `git config --global --list` contains the user's `user.name` and `user.email` values.

2.3 Installing PlatformIO IDE

PlatformIO IDE is integrated into Visual Studio Code through the official PlatformIO extension. This extension provides a development environment for embedded projects and is used to build, upload, and debug EduFramework projects.
Open Visual Studio Code, select **Extensions** on the Activity Bar, and search for `PlatformIO IDE`. Select the **PlatformIO IDE** extension published by **PlatformIO**, then select **Install** to begin the installation.

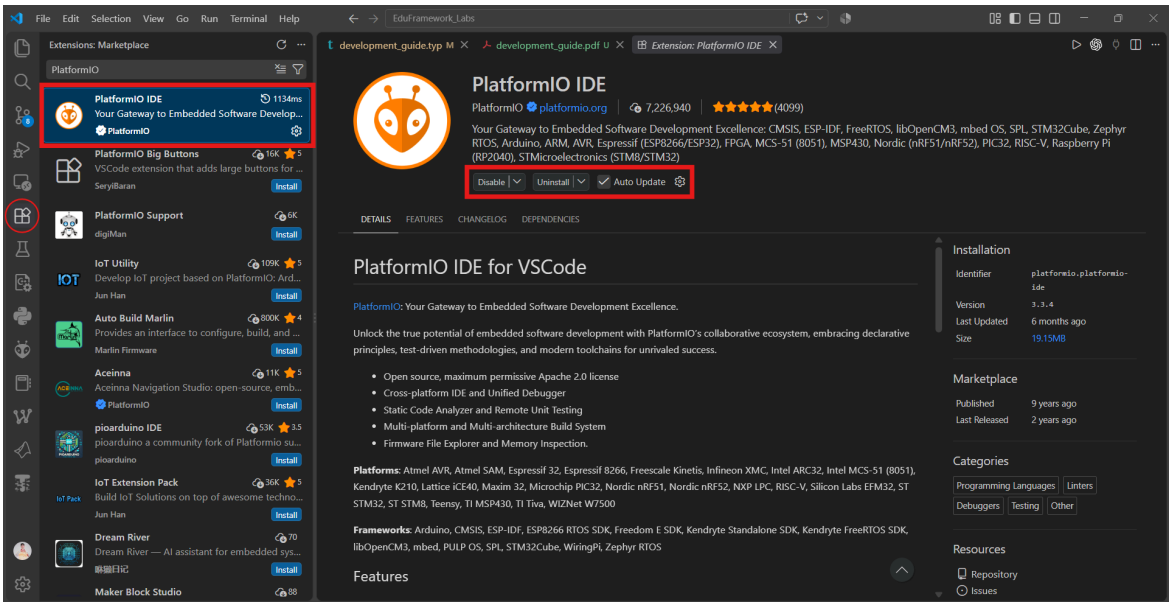

Figure 2.7. Installing PlatformIO IDE from the Visual Studio Code Extensions Marketplace

After the extension is installed, PlatformIO initializes the required components. This process may take some time during the first launch.

Note

During the first launch, wait for PlatformIO to complete initialization before continuing. The required time may vary depending on the system and network connection.

After initialization is complete, select the **PlatformIO** icon on the Activity Bar, then select **PIO Home → Open** to open PlatformIO Home.

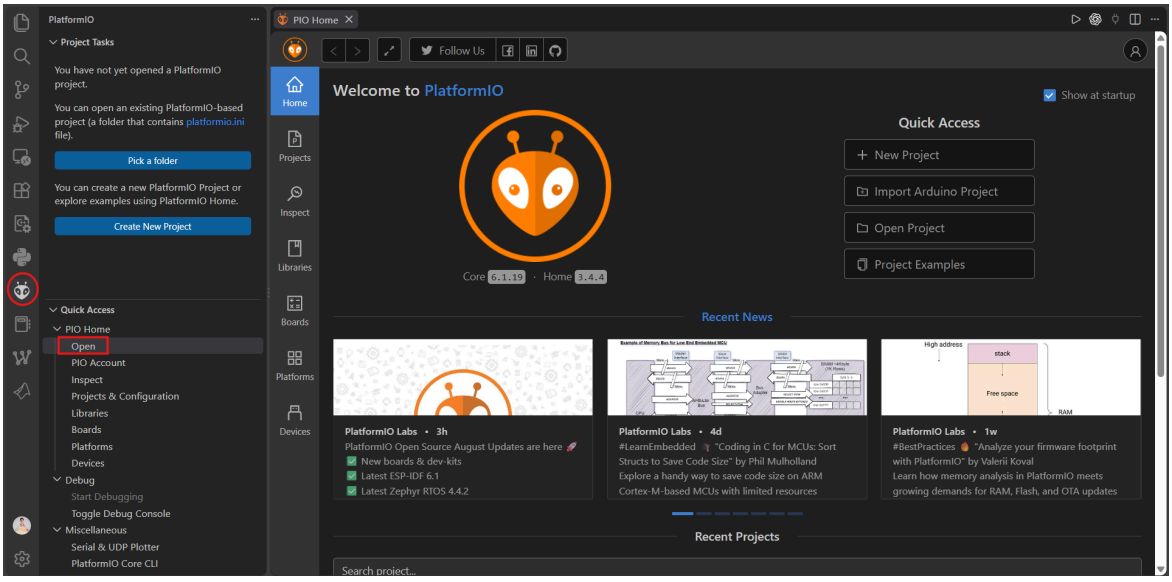

Figure 2.8. PlatformIO Home after successful installation

EXPECTED RESULT

PlatformIO IDE has been installed and initialized successfully. PlatformIO Home can be opened directly from Visual Studio Code.

3 Creating the First EduFramework Project

This section explains how to create a minimal EduFramework project for the S32K144, configure the project with PlatformIO, write the first program, and upload it to the MaaZEDU board.

3.1 Creating the Project Structure

In Visual Studio Code, select **File → Open Folder…**. Create a new project folder, for example `EduFramework_First_Project`, then open the newly created folder in Visual Studio Code.

In the Visual Studio Code Explorer, create the `src` directory, then create `main.c` inside it. In the project root directory, also create `platformio.ini`.

The initial project structure is as follows:

Cấu trúc project EduFramework

```
EduFramework_First_Project/
├── platformio.ini
└── src/
    └── main.c
```

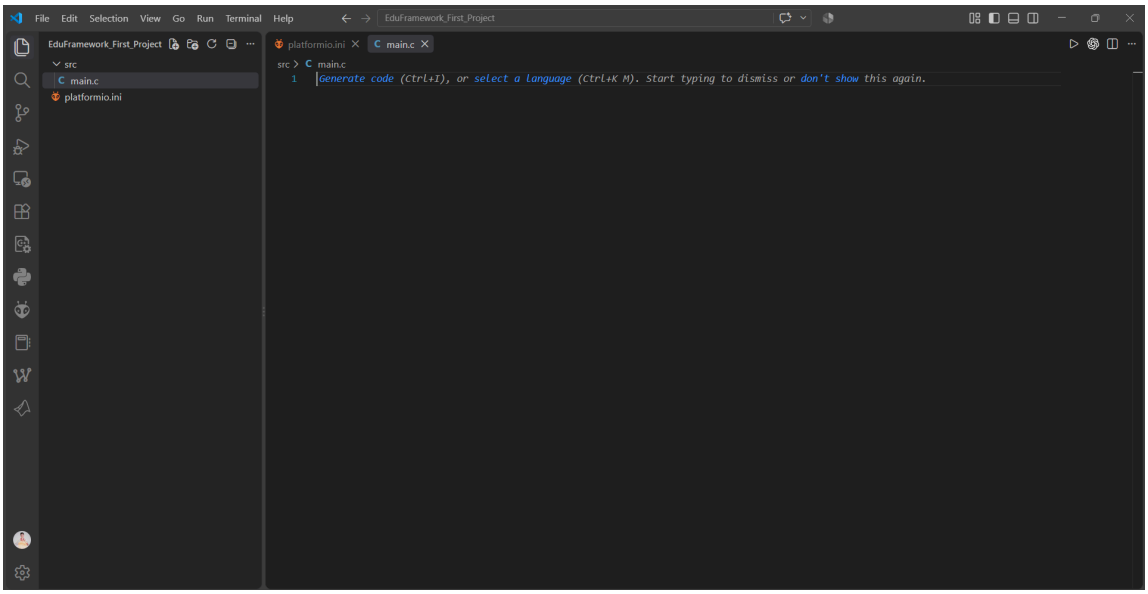

Figure 3.1. Initial structure of the EduFramework project

3.2 Opening the Project with PlatformIO

After creating the project structure, open PlatformIO Home using the PlatformIO icon on the Activity Bar. Select **PIO Home → Open → Open Project** to open the project.

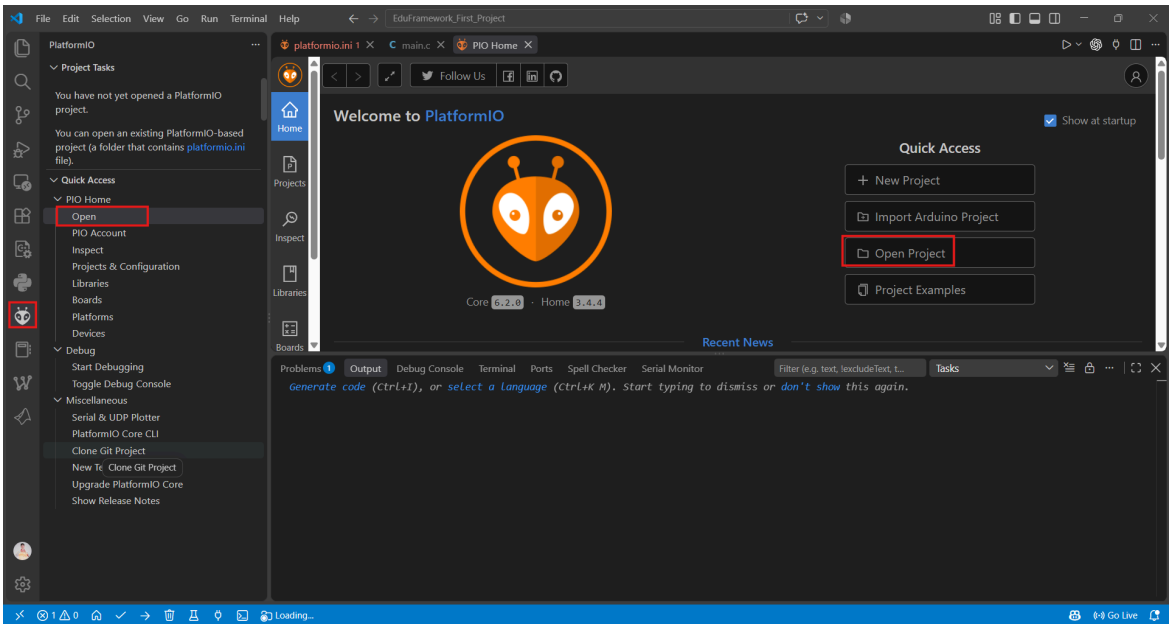

Figure 3.2. Opening the project from PlatformIO Home

Select the project folder that was just created. The selected folder must contain `platformio.ini` and the `src` directory.

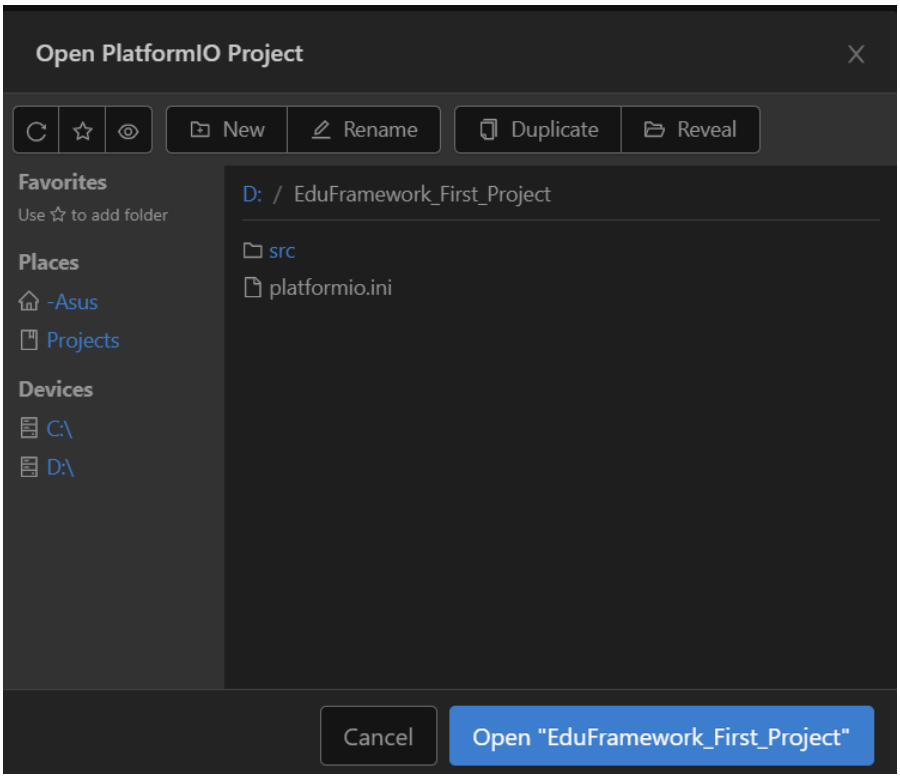

Figure 3.3. Selecting the EduFramework project folder

3.3 Configuring the Project

Open `platformio.ini` and define the S32K144 environment as follows:

```
[env:s32k144]
platform = https://github.com/QuangTM15/platform-nxps32k.git
board = s32k144
framework = eduframework
upload_protocol = jlink
debug_tool = jlink
```

Listing 3.1. PlatformIO configuration for the EduFramework project

In this configuration, `platform` specifies the Platform-NXPS32K platform used by the project, `board` identifies the target board, `framework` selects EduFramework, while `upload_protocol` and `debug_tool` configure J-Link for uploading and debugging.

After saving `platformio.ini`, PlatformIO recognizes the project configuration and prepares the development environment. On first use, this process may take some time because PlatformIO needs to download and install the required components.

PlatformIO-managed directories and files, such as `.pio` and `.vscode`, may appear in the project after initialization is complete.

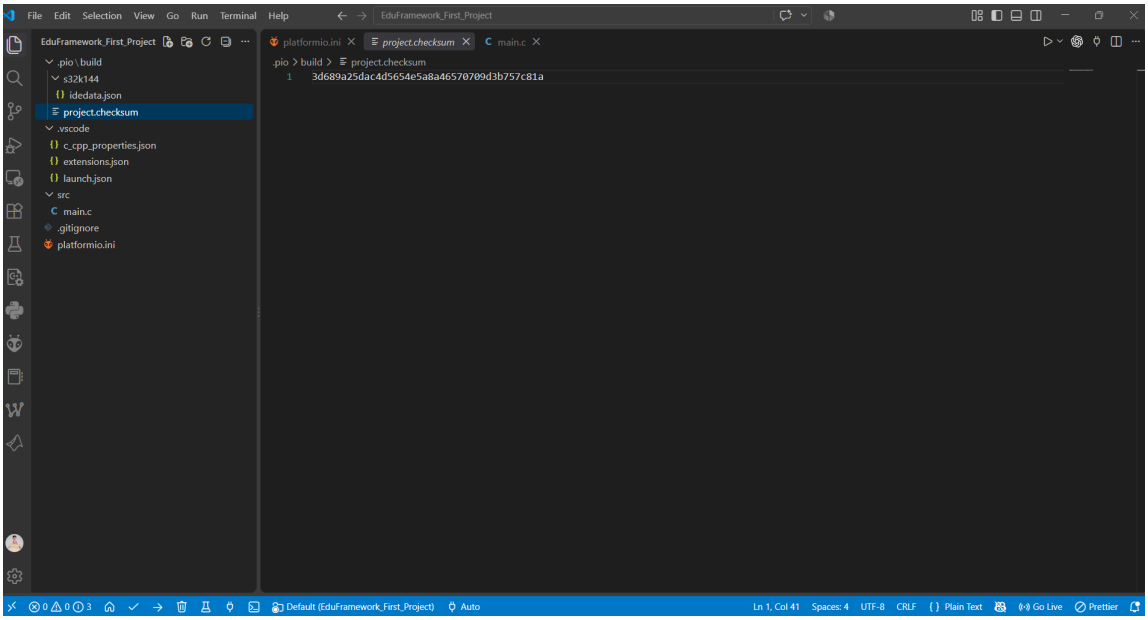

Figure 3.4. EduFramework project after PlatformIO initialization

Note

Files inside the `.pio` directory do not need to be edited manually. The contents of this directory are managed by PlatformIO and may be regenerated during the project build process.

3.4 Writing the First Program

Open `src/main.c` and enter the following program:

```c
#include "Arduino.h"

volatile int counter = 0;

int main(void)
{
    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        digitalWrite(LED_RED, LOW);
        delay(500U);

        digitalWrite(LED_RED, HIGH);
        delay(500U);

        counter++;
    }

    return 0;
}
```

Listing 3.2. First Blink LED program with EduFramework

The `setup()` function must be called before using any EduFramework API. This function performs the minimum initialization of hardware components and resources required for the framework to operate before the application continues execution.

> **WARNING**
>
> When developing an application with EduFramework, `setup()` must be called before any framework API is used. Omitting this initialization may cause the system to hang or result in unexpected behavior during program execution.

3.5 Building the Project

Before building, save all modified files.
Select the **Build** icon on the PlatformIO toolbar at the bottom of Visual Studio Code to compile the project.
When the build completes successfully, the terminal displays the `SUCCESS` status.

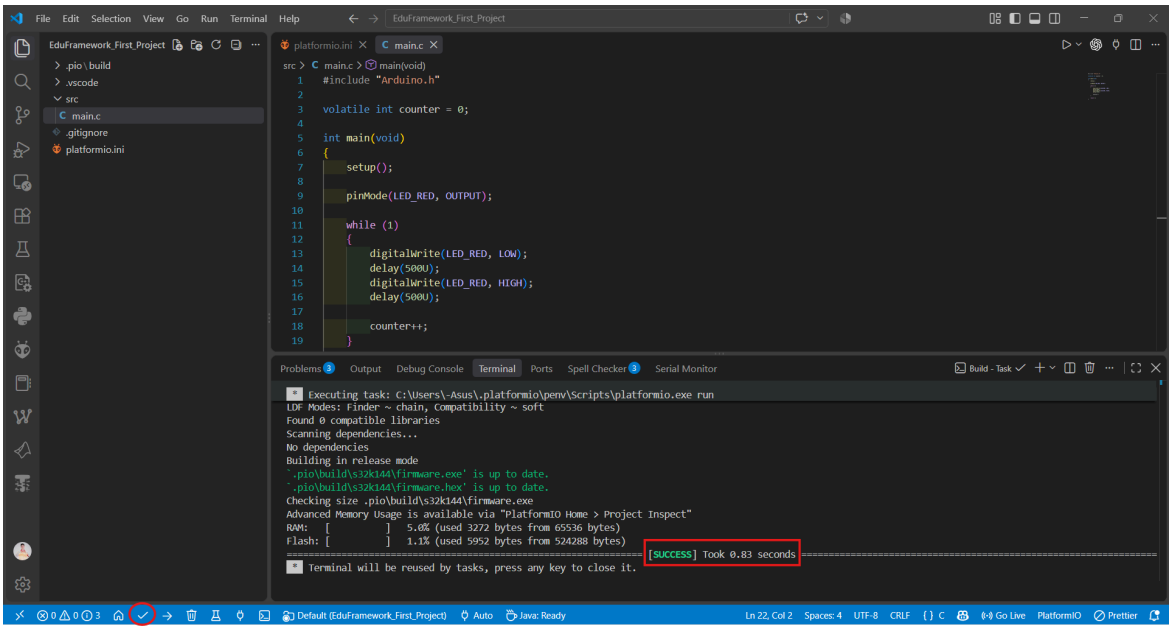

Figure 3.5. Successful EduFramework project build

3.6 Uploading the Program to the Board

Connect the MaaZEDU board to the computer using the USB port used for program upload and debugging. After the board is connected, select the **Upload** icon on the PlatformIO toolbar to upload the program to the S32K144.

PlatformIO uses the `upload_protocol = jlink` setting in `platformio.ini` to perform the upload. When the process completes successfully, the terminal displays the `SUCCESS` status.

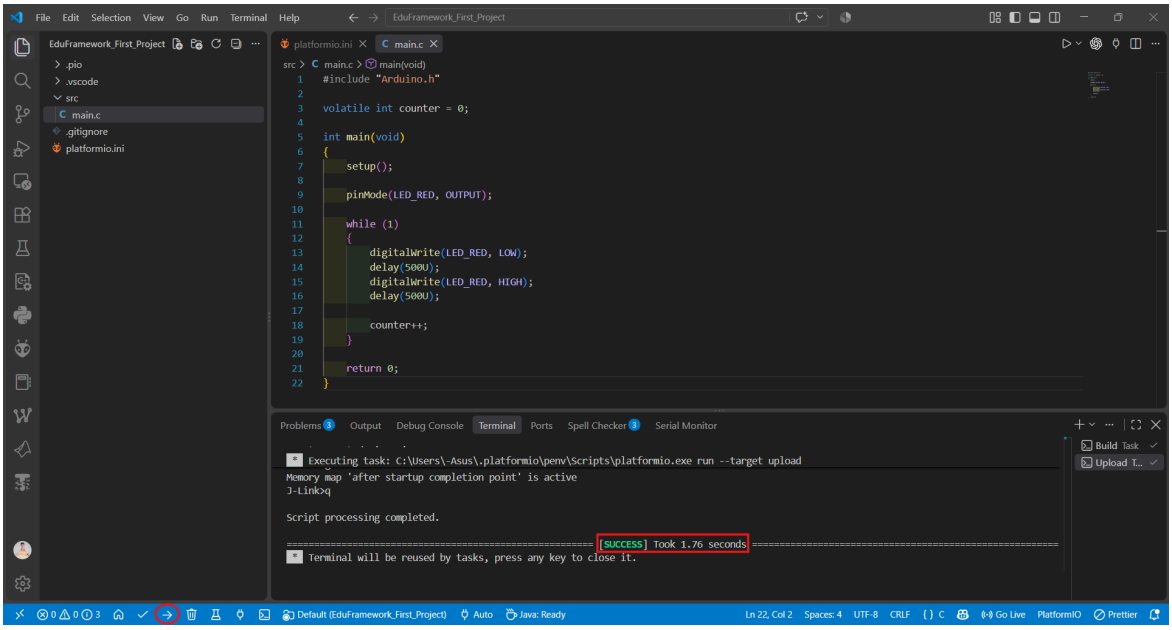

Figure 3.6. Successful program upload to the S32K144

4 Debugging

PlatformIO integrates a debugger directly into Visual Studio Code, allowing the program to be paused, executed step by step, and variable values to be monitored during execution. This section uses the project created previously to perform a basic debugging session on the S32K144.

4.1 Starting a Debug Session

Ensure that the MaaZEDU board remains connected to the computer through the USB port used for uploading and debugging.

In Visual Studio Code, select the PlatformIO icon on the Activity Bar. Under **Debug**, select **Start Debugging** to start a debugging session.

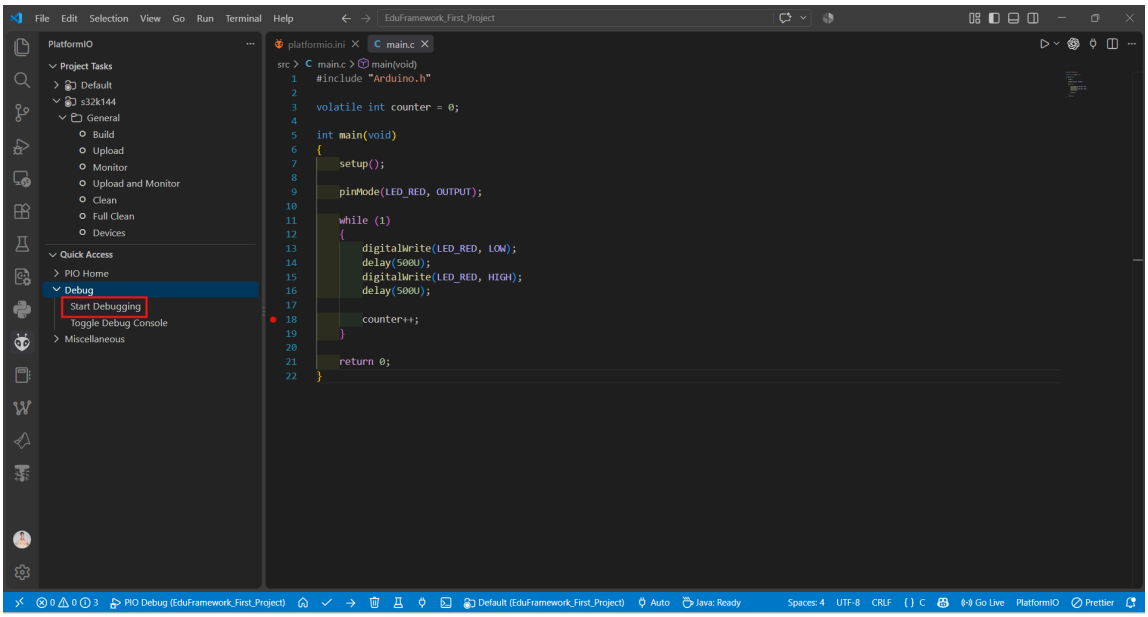

Figure 4.1. Starting a debugging session from PlatformIO

PlatformIO prepares the program, connects to the debugger, and starts the debugging session. When this process is complete, Visual Studio Code switches to the **Run and Debug** interface.

With the current configuration, the program may pause at the beginning of `main()` when the debugging session starts. From this point, the debug controls can be used to control program execution.

4.2 Debug Controls and Breakpoints

When a debugging session is active, the debug control toolbar appears at the top of the Visual Studio Code window.

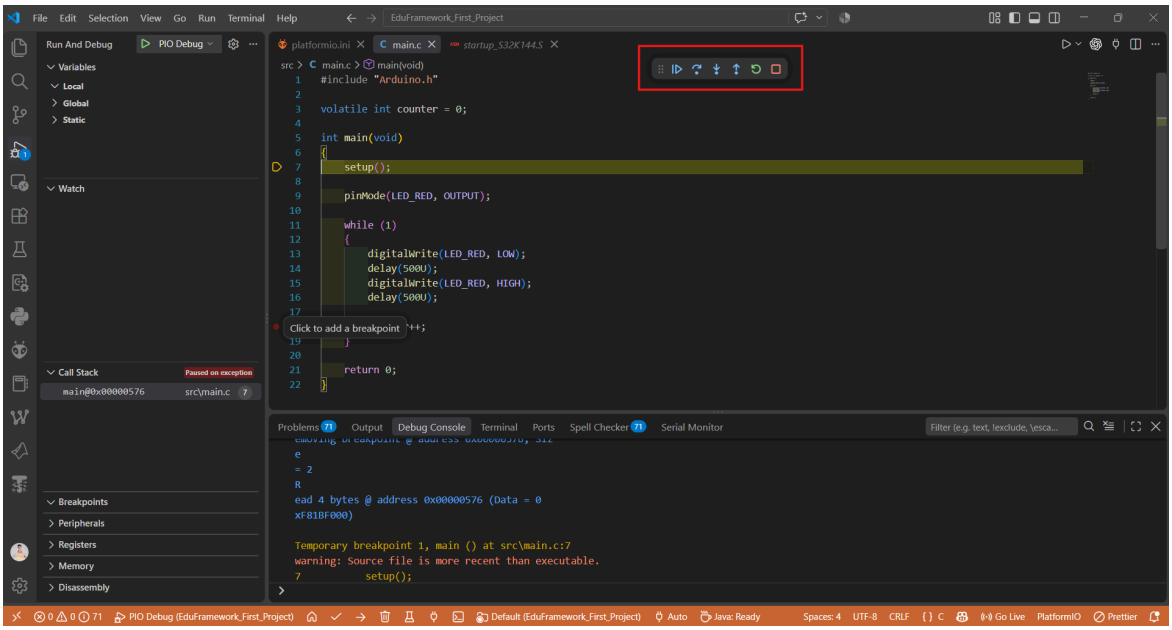

Figure 4.2. Debug interface, control toolbar, and breakpoint

The buttons on the debug control toolbar are arranged from left to right as follows:

Tool	Shortcut	Function
Continue	`F5`	Continues program execution until the next breakpoint or another condition causes execution to stop.
Step Over	`F10`	Executes the current line and moves to the next line without entering a function called on that line.
Step Into	`F11`	Enters the function called on the current line so that its instructions can be debugged step by step.
Step Out	`Shift + F11`	Continues execution until the current function returns to its caller.
Restart	`Ctrl + Shift + F5`	Restarts the debugging session from the beginning.
Stop	`Shift + F5`	Stops the current debugging session.

Table 4.1. Basic debug control tools

A breakpoint is a stopping point placed on a source-code line. When program execution reaches a line containing a breakpoint, the debugger pauses execution so that variables and program state can be inspected. To set a breakpoint, click in the gutter to the left of the desired source-code line. A red circle appears on that line. Click the same location again to remove the breakpoint.
In the current project, set a breakpoint on the following line:

```
counter++;
```

Listing 4.1. Statement used as the breakpoint location

4.3 Monitoring Variables with Watch

The **Watch** panel allows the value of a variable or expression to be monitored directly during debugging.
In the **Run and Debug** interface, open the **Watch** section, select the `+` icon, and enter:

Terminal

```
counter
```

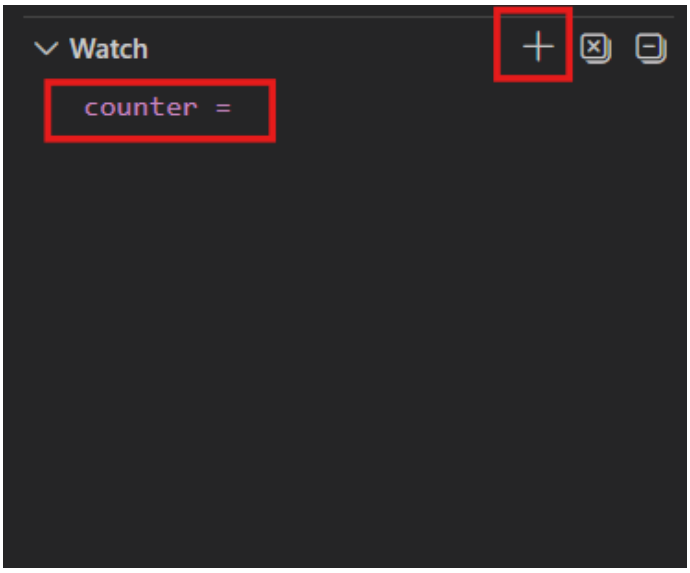

Figure 4.3. Adding the counter variable to the Watch panel

After it is added to Watch, the value of `counter` is updated whenever program execution pauses during the debugging session.

4.4 Continuing Program Execution and Observing the Result

After setting the breakpoint at `counter++` and adding `counter` to Watch, select **Continue** (`F5`) to resume program execution.

The program executes until it reaches the breakpoint at `counter++`, at which point the debugger pauses. The current value of `counter` can then be observed in the Watch panel.

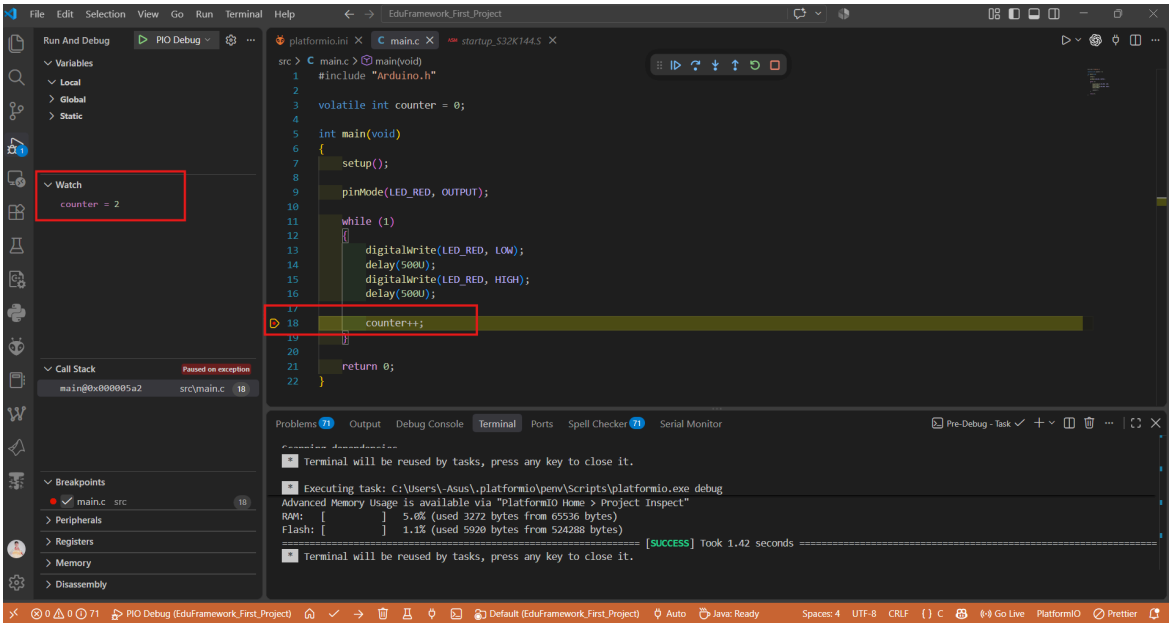

Figure 4.4. Observing the counter value at the breakpoint

Select **Continue** (`F5`) again to allow the program to run through another cycle. When the breakpoint is triggered again, the value of `counter` changes according to program execution.

The **Step Over**, **Step Into**, and **Step Out** tools can be used when a more detailed view of the execution flow is required at the current stopping point.

5 Troubleshooting

This section will be completed after the entire EduFramework setup and development workflow has been tested on a clean computer environment. Practical issues related to tool installation, PlatformIO initialization, build, upload, and debugging will be collected together with their corresponding solutions.

6 Next Steps

After completing the steps in this guide, the EduFramework development environment is ready to build, upload, and debug applications for the S32K144 through PlatformIO.
The EduFramework Laboratory Series continues by introducing framework functionality through topic-based laboratory exercises. The laboratories begin with fundamental functions such as Digital Output and Digital Input, then expand to peripherals and devices supported by EduFramework.
All laboratory documents, example projects, and related source code are maintained at:

> **GitHub Repository** `EduFramework Laboratory Series`

EduFramework and Platform-NXPS32K can be referenced directly in their respective repositories:

> **GitHub Repository** `EduFramework`

> **GitHub Repository** `Platform-NXPS32K`

7 References

[1] Git, *Git Documentation* .

[2] GitHub, *GitHub Docs* .

[3] Microsoft, *Visual Studio Code Documentation* .

[4] PlatformIO, *PlatformIO Documentation* .

[5] QuangTM15, *EduFramework for NXP S32K144* .

[6] QuangTM15, *Platform-NXPS32K* .

[7] QuangTM15, *EduFramework Laboratory Series* .